
Hardening funcional de una landing de *infoproducto*.

Cómo dejar lista para vivir una landing de venta digital — legal, técnicamente y comercialmente — replicando lo aplicado en arkwright.laralawn.com sin contratar Sentry, Plausible, GA4 ni UptimeRobot.

VERSIÓN

1.0

FECHA

14 MAYO 2026

STACK

NEXT 14 · PRISMA ·
STRIPE · RESEND ·
RAILWAY

TIEMPO TOTAL

≈ 3.5 DÍAS SOBRE UN
PROYECTO BASE

Índice

01.	Resumen ejecutivo	3
02.	Decisiones de diseño y filtros aplicados	4
03.	Stack asumido y variables de entorno	5
04.	Modelo de datos añadido (Prisma)	7
05.	Sprint 1 · Legal y consentimiento	8
06.	Sprint 2 · Operativo y SEO	10
07.	Sprint 3 · Conversión y self-service	12
08.	Mantenimiento día a día	14
09.	Checklist de replicación paso a paso	15
10.	Plantillas reutilizables	17
11.	Anexos · enlaces de verificación	20

Qué cubre este manual y qué no.

Este documento sirve para llevar cualquier landing de un infoproducto digital al mismo nivel de hardening que `arkwright.laralawn.com`: legal al día con normativa española y europea, telemetría interna funcionando, self-service de recibo, y observabilidad del propio servidor — todo sin contratar servicios externos nuevos.

ARCHIVOS CREADOS

18

de Sprint 1 a Sprint 3

ARCHIVOS MODIFICADOS

7

layout, footer, webhook...

SERVICIOS EXTERNOS
NUEVOS

0

filtro explícito del cliente

Lo que SÍ incluye

- **Sprint 1** · 4 páginas legales (aviso legal, privacidad, términos, cookies) y banner de consentimiento sin librerías externas.
- **Sprint 2** · Página de contacto + endpoint de salud + tracking de errores en BD + sitemap + robots + JSON-LD + OG image dinámica.
- **Sprint 3** · Stripe Customer Portal (recibo / método de pago / refund si decides activarlo) + tracking de conversión propio + dashboard `/admin/analytics`.

Lo que NO incluye (descartado por filtro)

- **Sentry / Rollbar / Bugsnag** → sustituido por tabla `ErrorLog` + `/admin/errors`.
- **Plausible / Google Analytics / Mixpanel** → sustituido por tabla `Event` + `/admin/analytics`.
- **UptimeRobot / Better Stack** → sustituido por `/api/health` + curl/cron manual.
- **Hotjar / Microsoft Clarity** → sin equivalente interno sencillo, queda fuera.
- **ConvertKit / Mailchimp para sequences** → fuera; Resend ya envía transaccionales pero sin automation built-in.

Filtro central del cliente: nada que requiera abrir una cuenta nueva en otro SaaS. Todo lo que se decide aquí cabe en Postgres y se administra desde `/admin`.

Por qué este manual mete todo dentro del propio Next.

Telemetría 100 % en Postgres

Cada visita y cada error vive en la misma base de datos que las compras. Ventaja: nunca pagas un plan por exportar tus datos, y los joins entre "evento" y "compra" son SQL directo. Desventaja: no hay UI de tableau-style out of the box — pero `/admin/analytics` ya cubre el 90 % del uso real (visits / día, embudo, top UTMs, top paths).

Cookie banner sin librería externa

El componente `CookieBanner.tsx` son ≈ 140 líneas, persiste el consentimiento en `localStorage.cookieConsent` versionado y con caducidad de 6 meses (cumple la guía de la AEPD). Expone `window.__cookieConsent` para que cualquier script de tracking lea el flag *analytics* antes de disparar.

Política española estricta por defecto

- Aviso legal con datos LSSI-CE obligatorios (razón social, NIF, domicilio, registro mercantil).
- Privacidad con tabla finalidad / base jurídica / conservación (RGPD UE 2016/679 + LOPDGDD 3/2018).
- Términos con **waiver explícito** del derecho de desistimiento (Art. 103.m RD-Ley 1/2007) — necesario para "no refunds" en producto digital con acceso inmediato.
- Política de cookies cumpliendo Art. 22.2 LSSI-CE.

JSON-LD Schema.org para AI search

Un solo componente `JsonLd.tsx` con dos exports: `JsonLdGlobal` (Organization + WebSite, inyectado en `layout.tsx`) y `JsonLdLanding` (Person + Book + Product con `aggregateRating`). Hoy ya posiciona en Rich Results. Mañana — cuando los LLMs sigan citando — sigue siendo la mejor inversión de 1 hora del proyecto.

Regla de oro: si una funcionalidad se puede resolver con Postgres + Next, vale más que un SaaS, aunque el SaaS tenga mejor UI. El coste no es la suscripción mensual — es la fragilidad de un dato que ya no controlas.

Lo mínimo para que esto se sostenga en producción.

Componentes obligatorios

Capa	Tecnología	Por qué
Framework	Next.js 14 App Router	RSC + edge + sitemap/robots/og nativos.
Lenguaje	TypeScript estricto	Evita 90 % de los bugs en runtime.
BD	PostgreSQL (Railway)	Backups automáticos diarios. JSON soportado.
ORM	Prisma 5+	<code>groupBy</code> y <code>\$queryRaw</code> resuelven el dashboard analytics.
Pagos	Stripe Checkout + Customer Portal	Cumplimiento PCI delegado. Portal sustituye al panel de soporte.
Email	Resend	Transaccional + dominio verificado. Sin pricing escalonado.
Hosting	Railway	Postgres incluido + auto-deploy desde GitHub + backups.

Variables de entorno

Generales

Variable	Para qué
<code>DATABASE_URL</code>	Conexión Prisma → Postgres (Railway la inyecta automática).
<code>NEXT_PUBLIC_SITE_URL</code>	Dominio canónico sin barra final. Ej: <code>https://arkwright.laralawn.com</code> .
<code>ADMIN_EXPORT_TOKEN</code>	Token largo aleatorio. Da acceso a <code>/admin</code> , <code>/admin/errors</code> , <code>/admin/analytics</code> y CSV export.

Stripe

Variable	Para qué
<code>STRIPE_SECRET_KEY</code>	Clave secreta live (<code>sk_live_...</code>).
<code>STRIPE_PRICE_ID</code>	Precio del producto digital. Crear en Stripe → Products.

`STRIPE_WEBHOOK_SECRET`

Firma del webhook (`whsec_...`) — Stripe la da al crear el endpoint.

Resend

Variable	Para qué
<code>RESEND_API_KEY</code>	API key (<code>re_...</code>) tras verificar dominio.
<code>RESEND_FROM</code>	Remitente. Ej: <code>"INNERAXIS <info@inneraxisinstitute.com>"</code> .
<code>RESEND_CONTACT_TO</code>	Buzón al que llegan los mensajes de <code>/contacto</code> .

Producción ≠ `.env.local` . Railway tiene su propio panel de variables. Comprueba siempre con `railway variables` o desde el dashboard antes de desplegar.

Dos tablas nuevas en Prisma. Nada más.

Estas dos tablas se añaden al `schema.prisma` existente. La migración se genera con `npx prisma migrate dev --name add_error_log_and_event`.

Tabla `ErrorLog`

```
model ErrorLog {
  id          String    @id @default(uuid()) @db.Uuid
  source      String    // "checkout" | "webhook" | "biblioteca" | ...
  message     String
  stack       String?
  path        String?
  userAgent   String?   @map("user_agent")
  ipHash      String?   @map("ip_hash")
  meta        Json?
  createdAt   DateTime  @default(now()) @map("created_at")

  @@index([createdAt])
  @@index([source, createdAt])
  @@map("error_logs")
}
```

Tabla `Event`

```
model Event {
  id          String    @id @default(uuid()) @db.Uuid
  kind        String
  path        String?
  sessionId   String?   @map("session_id")
  utmSource   String?   @map("utm_source")
  utmMedium   String?   @map("utm_medium")
  utmCampaign String?   @map("utm_campaign")
  referrerHost String?  @map("referrer_host")
  abVariant   String?   @map("ab_variant")
  meta        Json?
  createdAt   DateTime  @default(now()) @map("created_at")

  @@index([createdAt])
  @@index([kind, createdAt])
  @@map("events")
}
```

Helpers asociados

- `src/lib/logError.ts` → `logError({ source, error, path, userAgent, ip, meta })` persiste en `error_logs`. Best-effort: si falla la escritura, hace `console.error` y traga la excepción.
- `src/lib/events.ts` → `recordEvent({ kind, path, sessionId, utm*, meta })` persiste en `events`. Mismo patrón best-effort.

Ambas tablas son **append-only**. Para purgar, `DELETE FROM error_logs WHERE created_at < NOW()`
- `INTERVAL '90 days'` mensual, o un cron en Railway.

Legal y consentimiento.

Archivos a crear

Path	Qué hace
<code>src/app/legal/aviso-legal/page.tsx</code>	Datos LSSI-CE obligatorios.
<code>src/app/legal/privacidad/page.tsx</code>	Política RGPD con tabla finalidades.
<code>src/app/legal/terminos/page.tsx</code>	Términos + waiver Art. 103 RDL 1/2007.
<code>src/app/legal/cookies/page.tsx</code>	Tabla de cookies + botón reabrir banner.
<code>src/components/LegalLayout.tsx</code>	Wrapper con tipografía editorial y "Volver al inicio".
<code>src/components/CookieBanner.tsx</code>	Banner GDPR self-contained.

Datos a recolectar antes de empezar

- Denominación social (S.L. / S.A. / persona física).
- NIF / CIF.
- Dirección fiscal completa.
- Email legal de contacto.
- Tomo / folio / hoja del Registro Mercantil (si ya están inscritos — si no, dejar placeholder explícito).
- Política de devoluciones: ¿hay garantía o no? Esto decide cómo se redactan términos y FAQ.

Contrato del banner

Cuatro reglas que el código asume:

- El banner está **oculto** en `/admin/*` y `/legal/*` (no obstruye lo que el usuario está leyendo).
- Se persiste con versión (`CONSENT_VERSION`). Si cambias categorías, súbela y todos vuelven a ver el banner.
- Caducidad 6 meses. Cumple el plazo orientativo de la AEPD.
- Expone `window.__cookieConsent` globalmente para tracking client-side.

Modificaciones

- `src/components/Footer.tsx` · añadir 4 links legales + link a `/contacto` + copyright con razón social.
- `src/app/layout.tsx` · insertar `<CookieBanner />` antes de cerrar `<LangProvider>`.
- `src/app/globals.css` · ≈180 líneas para clases `.legal-*`, `.cookie-banner-*`, `.footer-legal`.

Si tu política dice "no refunds": el waiver del Art. 103.m debe estar *antes* de la pasarela de pago, y la FAQ debe responder honestamente que no se admiten devoluciones. Esconderlo invalida el waiver y te expone a reclamaciones del consumidor.

Operativo y SEO sin servicios externos.

Archivos a crear

Path	Qué hace
<code>src/app/contacto/page.tsx</code>	Formulario name + email + mensaje.
<code>src/app/api/contacto/route.ts</code>	POST → envía vía Resend, rate-limit 5/hora IP.
<code>src/app/api/health/route.ts</code>	GET 200/503 según Prisma, Stripe, Resend.
<code>src/app/admin/errors/page.tsx</code>	Lista últimos 200 errores, pills por source.
<code>src/app/sitemap.ts</code>	Genera <code>sitemap.xml</code> nativo.
<code>src/app/robots.ts</code>	Permite landing + legales, bloquea <code>/admin</code> <code>/biblioteca</code> .
<code>src/components/JsonLd.tsx</code>	Organization + WebSite + Person + Book + Product.
<code>src/app/opengraph-image.tsx</code>	1200×630 generada con <code>next/og</code> .
<code>src/app/twitter-image.tsx</code>	Igual que la anterior, expuesta como Twitter Card.

Salud del sistema

`/api/health` devuelve `200` con un objeto si Prisma responde a `SELECT 1`, Stripe tiene secret key configurada y Resend tiene API key. Devuelve `503` con el detalle del fallo en cualquier otro caso. Para chequear desde fuera:

```
curl -s https://arkwright.laralawn.com/api/health | jq
```

Si quieres notificación, programa un cron en Railway o un workflow de GitHub Actions que llame al endpoint cada 5 min y abra issue al fallar.

OG image dinámica

Generada con `next/og` + `ImageResponse`. Sin dependencias adicionales. Se sirve en `/opengraph-image` y se enlaza automáticamente por Next al declarar el archivo `opengraph-image.tsx` en App Router.

JSON-LD inyectado

- `JsonLdGlobal` en `layout.tsx` · Organization + WebSite. Cubre cualquier URL.
- `JsonLdLanding` en el componente de la landing · Person (autor) + Book + Product + aggregateRating.

Verifica antes de publicar: Google Rich Results Test (search.google.com/test/rich-results)
acepta la URL de desarrollo o producción y te dice qué tipos detecta. Si te marca un fallo en *Product*,
suele ser porque falta `offers.priceCurrency` o `offers.availability` .

Conversión y self-service.

Stripe Customer Portal

El endpoint `POST /api/stripe/portal` recibe `{ token }` con el `accessToken` del Purchase (el mismo de `/biblioteca`), obtiene la sesión de Stripe, recupera (o crea si fue invitado) el Customer, abre una sesión temporal del portal y devuelve `{ url }`. El front en `BibliotecaHero.tsx` tiene un botón "Ver mi recibo" que llama al endpoint y redirige.

Toggle a revisar: Stripe Dashboard → Settings → Customer Portal → "Allow customers to request refunds". Si tu política es "no refunds", desmárcalo. Si decides activarlo, el webhook ya escucha `charge.refunded` y marca `Purchase.status = "refunded"`.

Tracking de conversión interno

Cinco kinds de evento, validados server-side en `/api/track`:

Kind	Cuándo se dispara	Subjeto a consent
<code>page_load</code>	Cada cambio de pathname client-side.	Sí
<code>cta_click</code>	Click en <code>[data-track-cta]</code> que NO sea <i>checkout</i> .	Sí
<code>checkout_start</code>	Click en <code>[data-track-cta="checkout"]</code> .	Sí
<code>purchase_completed</code>	Webhook <code>checkout.session.completed</code> .	No (legítimo interés)
<code>refund_requested</code>	Webhook <code>charge.refunded</code> .	No

Garantías del tracker

- El componente `Tracker` NUNCA registra si `window.__cookieConsent.analytics !== true`.
- Usa `navigator.sendBeacon` con fallback a `fetch keepalive`. Best-effort: nunca espera respuesta.
- Genera `sessionId` con `crypto.randomUUID()` en `sessionStorage`. Caduca al cerrar pestaña — no identifica personas.
- UTM y referrer host se sniffean una vez por sesión y persisten.
- Rate-limit silencioso en el servidor: 60 eventos/min por IP. Por encima, devuelve `200` sin grabar.

Cómo etiquetar tus CTAs

En cualquier botón / link de conversión, añade `data-track-cta="label"`. El tracker hace event delegation con `closest()`, no hace falta pasar callbacks.

```
<button data-track-cta="checkout"> Comprar </button>
<a href="/oferta" data-track-cta="hero-link"> Más info </a>
```

Dashboard `/admin/analytics`

Las métricas que ya devuelve sin tocar nada:

- Big numbers · page_load · cta_click · checkout_start · purchase_completed (+ refund_requested si hay).
- Embudo · ratio visit → checkout y checkout → purchase.
- Serie diaria de visitas (últimos 30 días) como bar chart CSS.
- Top paths, top utm_source / medium / campaign, top referrer hosts.
- Breakdown por `abVariant` si lo usas para A/B test.

Lo que hay que mirar cada cuanto.

Frecuencia	Acción	URL / Comando
Diario (1 min)	Revisar <code>/admin/errors</code>	<code>/admin/errors?token=...</code>
Diario (30 s)	Health check	<code>curl /api/health</code>
Semanal (5 min)	Revisar <code>/admin/analytics</code> · rango 7d	<code>/admin/analytics?token=...&range=7d</code>
Semanal	Verificar último backup Postgres en Railway	Railway → Postgres → Backups
Mensual	Purgar errores antiguos	<code>DELETE FROM error_logs WHERE created_at < NOW() - INTERVAL '90 days'</code>
Mensual	Revisar Stripe Disputes	Stripe Dashboard → Disputes
Trimestral	Releer páginas legales — actualizar si cambia algo (cobro, contacto, finalidades)	<code>/legal/*</code>
Anual	Validar Schema.org y OG con Rich Results Test	<code>search.google.com/test/rich- results</code>

Buen hábito: guardar el token de admin en un password manager. Marca los bookmarks de `/admin/errors` y `/admin/analytics` con el token incluido en la URL. Así no tienes fricción para abrirlos cada mañana.

Aplicar este manual a otra landing en orden.

☐ Por hacer · ☒ Hecho. Imprime esta página y tachalo manualmente.

Fase 0 · Preparación

- ☐ Recoger denominación social, NIF/CIF, domicilio, email legal.
- ☐ Decidir política de devoluciones (con / sin garantía).
- ☐ Verificar dominio en Resend.
- ☐ Crear Stripe Live keys y producto + precio.
- ☐ Crear webhook endpoint `/api/stripe/webhook` en Stripe → guardar `STRIPE_WEBHOOK_SECRET`.
- ☐ Generar `ADMIN_EXPORT_TOKEN` aleatorio largo (32+ chars).

Fase 1 · Schema y env

- ☐ Añadir modelos `ErrorLog` y `Event` a `schema.prisma`.
- ☐ `npx prisma migrate dev --name add_error_log_and_event`.
- ☐ Crear `src/lib/logError.ts` y `src/lib/events.ts`.
- ☐ Subir env vars al panel del hosting (Railway / Vercel / etc.).

Fase 2 · Legal + consent (Sprint 1)

- ☐ Crear 4 páginas `/legal/*` con datos del cliente.
- ☐ Crear `LegalLayout` + `CookieBanner`.
- ☐ Añadir links legales al Footer + actualizar copyright.
- ☐ Pegar `<CookieBanner />` al final del `layout.tsx`.
- ☐ Verificar caducidad 6 meses y categorías necesarias/analytics/marketing.

Fase 3 · Operativo + SEO (Sprint 2)

- ☐ `/contacto` + `/api/contacto` con Resend.
- ☐ `/api/health` probado con `curl` y devuelve 200.

☐ `/admin/errors` gated por `ADMIN_EXPORT_TOKEN` .

☐ `sitemap.ts` + `robots.ts` publicados.

☐ `JsonLdGlobal` en layout, `JsonLdLanding` en la landing.

☐ `opengraph-image.tsx` + `twitter-image.tsx` .

☐ Validar con Rich Results Test → 0 errores.

Fase 4 • Conversión + portal (Sprint 3)

☐ Activar Stripe Customer Portal en el dashboard.

☐ Configurar el toggle de refund según política.

☐ `/api/stripe/portal` en producción.

☐ Botón "Ver mi recibo" en la página post-compra.

☐ `/api/track` con rate-limit y whitelist de kinds.

☐ Insertar `<Tracker />` en `layout.tsx` .

☐ Etiquetar todos los CTA con `data-track-cta="..."` .

☐ Modificar webhook para registrar `purchase_completed` y `refund_requested` .

☐ `/admin/analytics` renderiza correctamente con datos reales.

Fase 5 • Verificación end-to-end

☐ Compra de prueba con tarjeta Stripe `4242 4242 4242 4242` .

☐ Email transaccional recibido vía Resend.

☐ Acceso a `/biblioteca?token=...` .

☐ Botón "Ver mi recibo" abre Stripe Portal y vuelve.

☐ Evento `purchase_completed` aparece en `/admin/analytics` .

☐ Banner cookies: aceptar todo → POST `/api/track` registra `page_load` .

☐ Banner cookies: solo necesarias → 0 POST a `/api/track` .

Copy y JSON que puedes pegar tal cual.

Datos fiscales (rellenar antes de cualquier cosa)

```
Denominación social: _____  
NIF / CIF: _____  
Domicilio: _____  
Email legal: _____  
Inscripción RM: Tomo ____ Folio ____ Hoja ____
```

Cookie banner • copy ES

```
Usamos cookies necesarias para que el pago y el workbook  
funcionen. Las analíticas/marketing solo se activan si las  
aceptas. Más info en la política de cookies.
```

```
[Solo necesarias] [Aceptar todo]
```

Cookie banner • copy EN

```
We use necessary cookies to make checkout and the workbook  
work. Analytics/marketing only run if you accept them.  
More info in our cookie policy.
```

```
[Only necessary] [Accept all]
```

Cláusula waiver Art. 103 RDL 1/2007 (sin garantía de devolución)

```
Al iniciar la descarga / acceso inmediato al contenido digital,  
el usuario consiente expresamente la ejecución del contrato y  
RENUNCIA al derecho de desistimiento conforme al artículo 103.m)  
del Real Decreto Legislativo 1/2007, de 16 de noviembre, por el  
que se aprueba el texto refundido de la Ley General para la  
Defensa de los Consumidores y Usuarios.
```

JSON-LD • Organization (en `layout.tsx`)

```
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "INNERAXIS S.L.",
  "url": "https://arkwright.laralawn.com",
  "logo": "https://arkwright.laralawn.com/lara-lawn-logo.png",
  "email": "info@inneraxisinstitute.com",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "Calle Ruiseñor 22",
    "addressLocality": "Olías del Rey",
    "addressRegion": "Toledo",
    "postalCode": "45280",
    "addressCountry": "ES"
  }
}
```

JSON-LD • Product con rating (en la landing)

```
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "El método Arkwright",
  "description": "...",
  "image": "https://.../og-image.jpg",
  "brand": { "@type": "Brand", "name": "Lara Lawn" },
  "offers": {
    "@type": "Offer",
    "price": "12.00",
    "priceCurrency": "EUR",
    "availability": "https://schema.org/InStock",
    "url": "https://arkwright.laralawn.com/oferta"
  },
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingValue": "4.9",
    "ratingCount": "10000"
  }
}
```

robots.txt mínimo

```
User-agent: *
Allow: /
Allow: /oferta
Allow: /legal/
Disallow: /admin
Disallow: /biblioteca
Disallow: /workbook
Disallow: /registro
Disallow: /api/

Sitemap: https://arkwright.laralawn.com/sitemap.xml
```

sitemap.ts skeleton

```
import type { MetadataRoute } from "next";

const BASE = (process.env.NEXT_PUBLIC_SITE_URL ?? "")
  .replace(/\/$/, "");

export default function sitemap(): MetadataRoute.Sitemap {
  const now = new Date();
  const urls = [
    "", "/oferta", "/contacto",
    "/legal/aviso-legal", "/legal/privacidad",
    "/legal/terminos", "/legal/cookies",
  ];
  return urls.map((u) => ({
    url: `${BASE}${u}`,
    lastModified: now,
    changeFrequency: "monthly",
    priority: u === "" ? 1.0 : 0.7,
  }));
}
```

SQL útiles · purga y debug

```
-- Purgar errores de más de 90 días
DELETE FROM error_logs WHERE created_at < NOW() - INTERVAL '90 days';

-- Conversion rate últimos 30 días
SELECT
  COUNT(*) FILTER (WHERE kind = 'page_load')          AS visits,
  COUNT(*) FILTER (WHERE kind = 'checkout_start')     AS checkouts,
  COUNT(*) FILTER (WHERE kind = 'purchase_completed') AS purchases
FROM events
WHERE created_at >= NOW() - INTERVAL '30 days';

-- Top UTM source / campaign
SELECT utm_source, utm_campaign, COUNT(*) AS total
FROM events
WHERE created_at >= NOW() - INTERVAL '30 days'
  AND utm_source IS NOT NULL
GROUP BY 1, 2
ORDER BY total DESC
LIMIT 20;
```

Enlaces y referencias regulatorias.

Verificación técnica

- **Google Rich Results Test** · search.google.com/test/rich-results
- **Stripe webhook simulator** · Stripe Dashboard → Developers → Webhooks → tu endpoint → "Send test webhook"
- **Resend logs** · resend.com → Logs (ver delivery de transaccionales)
- **Railway backups** · Railway → Postgres service → Backups (verificar daily)

Normativa española / europea

- **LSSI-CE** · Ley 34/2002 de Servicios de la Sociedad de la Información y Comercio Electrónico. Art. 10 (información obligatoria) y Art. 22.2 (cookies).
- **RGPD UE 2016/679** · derechos del interesado, principio de minimización, base jurídica.
- **LOPDGDD 3/2018** · adaptación española del RGPD.
- **RDL 1/2007** · Ley General de Consumidores y Usuarios. Art. 103.m (waiver de desistimiento en contenido digital).
- **AEPD** · agencia española de protección de datos → guía cookies, plantillas de privacidad, mecanismos de reclamación.

Stripe

- **Customer Portal config** · Settings → Customer Portal → ajustar branding, return URL y toggles (refunds, cancel subscription, etc.).
- **Test cards** · 4242 4242 4242 4242 aprobada, 4000 0000 0000 0002 rechazada.
- **Webhook signing secret** · única forma segura de verificar que el evento viene de Stripe. NUNCA hardcodear; debe vivir en env.

Este manual es interno de INNERAXIS S.L. y refleja el estado de `arkwright.laralawn.com` el 14 de mayo de 2026. Cuando cambies algo del stack (nueva tabla, nuevo servicio, nueva normativa), actualízalo aquí antes de replicarlo a otra landing — la consistencia entre proyectos vale más que un manual perfecto en uno solo.



Cualquier landing que pase este manual completo es legal,
medible y mantenible. Sin más SaaS, sin más dashboards
perdidos, sin más facturas mensuales de servicios que casi no
usas.